

첫 패치가 머지되기까지

보낸 뒤 16일 동안 일어난 일과 확인하는 방법

2편에서 보낸 첫 패치가 media 트리의 next 브랜치에 들어갔다. 이 문서는 패치를 보낸 뒤 받아들여지기까지 무슨 일이 어떤 순서로 일어났는지, 그리고 그것을 메일과 patchwork, git으로 어떻게 확인했는지를 기록한 것이다. 보낸 쪽에서 할 일은 거의 없었다. 대신 무엇을 봐야 “머지됐다”고 말할 수 있는지를 정리했다.

2026년 9월 14--29일 기록. 대상 패치: media: Documentation: vivid.rst: Remove stale capture overlay mentions

1. 출발점

1.1 보낸 것

9월 14일 새벽에 보낸 패치는 Documentation/admin-guide/media/vivid.rst의 두 줄을 고친 것이다. vivid에서 capture overlay가 빠졌는데 문서 도입부와 기능 목록에는 여전히 남아 있던 것을 output overlay만 적도록 바꿨다.

```
Documentation/admin-guide/media/vivid.rst | 4 ++--  
1 file changed, 2 insertions(+), 2 deletions(-)
```

2편 마지막에 적은 대로, 리뷰는 보통 2-3주 안에 오고 재촉은 최소 1주일을 기다린 뒤에 한다. 그 사이에 3편과 4편의 패치 두 개를 더 보냈고, 이 패치에는 따로 손대지 않았다.

2. 시간 순서

메일, patchwork, git에 남은 시각을 모아 한국 시각으로 맞췄다. 커밋 시각은 커밋에 적힌 +0200 시각을 변환한 것이다.

```
09-14 05:24 sent (git send-email, Result: 250)  
09-14 08:17 patchwork CI: 12 checks, all success  
09-23 18:59 patchwork: New -> Under Review  
09-23 21:54 Hans Verkuil commits it (committer date 14:54 +0200)  
09-24 00:59 patchwork: Under Review -> Accepted  
09-27 06:17 media-ci mail: "Patch committed."  
09-29      checked lore, patchwork, origin/next
```

보낸 지 3시간 만에 자동 검사가 끝났고, 상태가 처음 바뀐 것은 9일 뒤였다. 그 뒤로는 하루 안에 커밋과 상태 변경이 끝났다.

3. 자동 검사

linuxtv의 patchwork는 패치가 올라오면 자동 검사를 돌리고 결과를 패치에 붙인다. 웹 화면 대신 patchwork의 API로 결과를 받아 봤다.

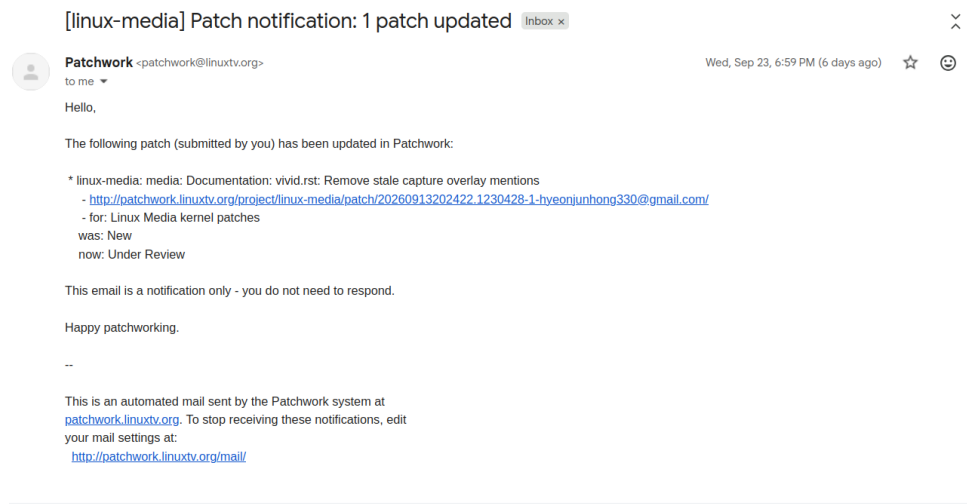
```
$ curl -s https://patchwork.linuxtv.org/api/patches/159301/checks/ \
  | jq -r '.[ ] | "\(.context) \(.state) "'
HTML_report success
report success
bisect success
doc success
build-basic success
cocci success
static success
abi success
media-patchstyle success
1-media_style success
checkpatch success
1-checkpatch success
```

12개 모두 통과였다. 문서만 고친 패치라서 doc(문서 빌드)이 가장 의미 있는 항목이다. 빌드, 정적 분석, 스타일 검사도 같이 돈다. 보내기 전에 로컬에서 checkpatch와 문서 빌드 전후 비교를 해 둔 것과 같은 종류의 검사를 서버가 한 번 더 한 셈이다. 여기서 실패하면 메인테이너가 보기 전에 걸러질 수 있으므로, 보낸 다음 날 이 결과부터 확인하는 것이 좋다.

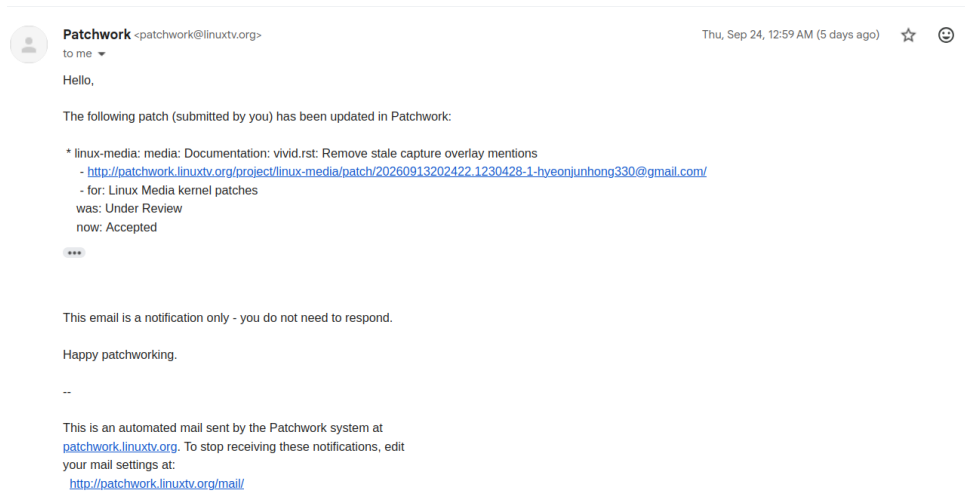
4. 알림 메일 세 통

4.1 patchwork 상태 변경

patchwork는 보낸 사람에게 상태가 바뀔 때마다 알림을 보낸다. 처음 온 것이 Under Review였다. 메인테이너가 이 패치를 집어 들었다는 뜻이다. patchwork의 담당자(delegate)도 hverkuil로 잡혀 있었다.



9월 23일. New에서 Under Review로 바뀌었다는 알림.

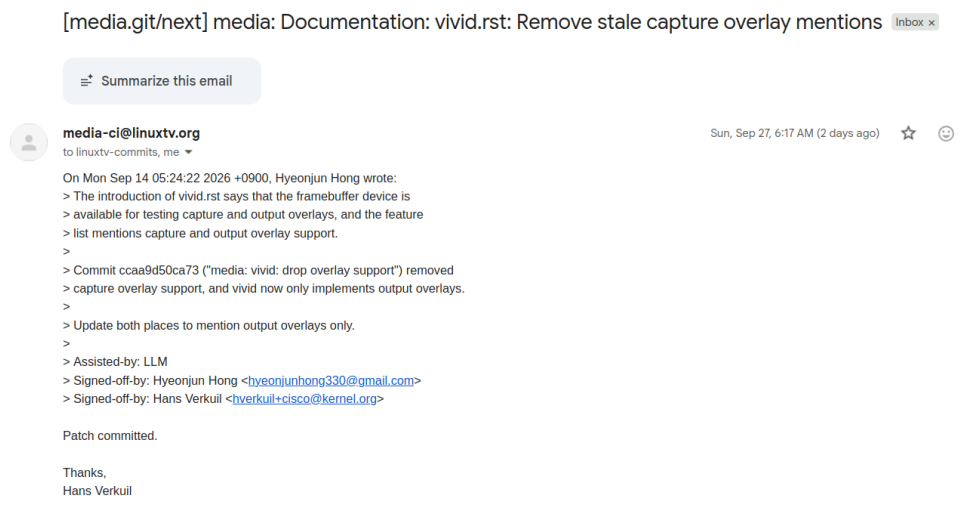


9월 24일. Under Review에서 Accepted로 바뀌었다는 알림.

Accepted는 “메인테이너가 자기 트리에 받아들였다”는 표시다. 다만 patchwork의 상태는 사람이 누르는 표시일 뿐이다. 실제로 트리에 들어갔는지는 git으로 따로 확인해야 한다.

4.2 커밋 알림

세 번째 메일은 patchwork가 아니라 media-ci@linuxtv.org에서 왔다. 받는 사람이 linuxtv-commits 리스트와 나로 되어 있다. 원래 커밋 메시지를 인용하고 그 아래에 “Patch committed.”와 메인테이너 이름, 그리고 실제로 적용된 diff 전체가 붙어 있다.



9월 27일. 제목의 [media.git/next]가 들어간 브랜치를 알려 준다.

이 메일은 답장을 기대하는 메일이 아니다. 감사 인사를 리스트에 보내면 받는 사람 모두에게 불필요한 메일이 하나 더 가므로 보내지 않았다.

4.3 리스트에는 답장이 없었다

lore의 스레드를 다시 받아 보니 메시지는 내가 보낸 원문 하나뿐이었다. 누구도 리스트에 “Reviewed-by”나 의견을 달지 않았고, 메인테이너는 답장 없이 적용했다. 리스트에 답장이 없는 것과 받아들여지지 않은 것은 다른 일이었다.

5. git으로 확인하기

5.1 트리에 있는가

작업 트리에서 원격을 새로 받고 내 이름으로 된 커밋을 찾았다.

```
$ git fetch origin
$ git log origin/next --oneline --author=hyeonjunhong330
87090658cf2f media: Documentation: vivid.rst: Remove stale capture overlay mentions
$ git branch -r --contains 87090658cf2f
origin/HEAD -> origin/next
origin/next
```

로컬에는 같은 제목의 커밋이 하나 더 있다(178ca060800e). 내가 만들어서 보낸 원본이다. 해시가 다른 것은 정상이다. 커밋 해시는 내용뿐 아니라 커밋한 사람, 시각, 부모 커밋까지 넣어 계산하므로, 메인테이너가 자기 트리에 적용하는 순간 새 해시가 생긴다. 앞으로 인용할 해시는 87090658cf2f다.

5.2 내용이 바뀌었는가

메인테이너가 적용하면서 손을 대는 경우가 있다. 오타를 고치거나 메시지를 다듬기도 한다. 그래서 보낸 패치 파일과 적용된 커밋을 비교했다. diff 내용만으로 계산하는 patch-id가 같으면 코드 변경은 한 글자도 다르지 않다는 뜻이다.

```
$ git show 87090658cf2f | git patch-id --stable
3dab2eb138a287d88662a3ff5ec96c654bac3892 87090658cf2f...
$ git patch-id --stable < outgoing/0001-media-Documentation-vivid.rst-*.patch
3dab2eb138a287d88662a3ff5ec96c654bac3892 178ca060800e...
```

두 값이 같았다. 커밋 메시지도 비교해 보니 달라진 것은 맨 끝의 한 줄뿐이었다.

```
Assisted-by: LLM
Signed-off-by: Hyeonjun Hong <hyeonjunhong330@gmail.com>
Signed-off-by: Hans Verkuil <hverkuil+cisco@kernel.org>
```

메인테이너가 자기 Signed-off-by를 붙였다. 이 줄은 “이 변경을 내 트리로 받아 위로 올려 보낸다”는 서명이다. 패치가 거쳐 간 사람이 순서대로 쌓이므로, 나중에 이 커밋을 보면 누가 쓰고 누가 받았는지 알 수 있다. 2편에서 고민했던 Assisted-by 태그는 그대로 남았다.

5.3 작성자와 커밋한 사람

```
$ git show -s --format='author %an %ad%ncommitter %cn %cd' 87090658cf2f
author Hyeonjun Hong Mon Sep 14 05:24:22 2026 +0900
committer Hans Verkuil Wed Sep 23 14:54:49 2026 +0200
```

git은 두 사람을 따로 기록한다. 작성자(author)는 패치를 쓴 사람이고 시각도 내가 보낸 때 그대로다. 커밋한 사람(committer)은 트리에 넣은 메인테이너다. 커널에서 “패치가 머지됐다”고 할 때 기록으로 남는 것은 작성자 쪽이다.

5.4 한 번에 같이 들어간 패치들

커밋 바로 아래를 보면 메인테이너가 일하는 방식이 보인다.

```
$ git log --first-parent --date=format:'%m-%d %H:%M' \
--format='%h %cd %an | %s' 87090658cf2f~7..87090658cf2f
87090658cf2f 09-23 14:54 Hyeonjun Hong | media: Documentation: vivid.rst: ...
22f552ff726a 09-23 14:54 Myeonghun Pak | media: cx23885: unregister DVB bus ...
9887716132d6 09-23 14:54 Myeonghun Pak | media: marvell: manage the MMP camera ...
359db09cf055 09-23 14:54 Ruoyu Wang | media: adv748x: Fix endpoint reference ...
c9c245a186ba 09-23 14:54 Hans Verkuil | media: i2c/platform: add V4L2_SET_EDID...
9ff738f93084 09-23 14:54 Rohinthan P | media: v4l2-ctrls: do nothing if ...
a601427f03e0 09-23 14:54 Pengpeng Hou | media: cec: extron: clear response tail ...
```

일곱 개의 커밋 시각이 모두 같은 분이다. 메인테이너는 패치를 하나씩 받는 것이 아니라 patchwork에서 여러 개를 모아 한꺼번에 적용한다. 서로 다른 드라이버, 다른 사람의 패치가 한 묶음으로 들어갔다.

6. 앞으로 남은 단계

6.1 next에서 메인라인까지

next는 다음 릴리스를 위해 모아 두는 브랜치다. 지금 이 브랜치의 Makefile은 7.3.0-rc1을 가리킨다. media 서브시스템은 머지 윈도우가 열리면 next에 모인 것을 Linus의 트리로 보낸다. 그 단계를 지나야 메인라인 커밋이 되고, 릴리스 태그에 포함된다.

순서대로라면 7.3이 나온 뒤 열리는 머지 윈도우에서 올라갈 것이다. 이것은 일반적인 흐름에서 나온 예상이고 아직 확인한 사실은 아니다. 메인라인에 들어갔는지는 그때 이렇게 확인한다.

```
$ git fetch torvalds
$ git log torvalds/master --oneline --author=hyeonjunhong330
$ git describe --contains <mainline hash> # first tag that includes it
```

next에 있는 동안에도 커밋이 되돌려질 수는 있다. 그래서 지금 기록에 남길 때는 “media 트리 next에 들어감”으로 적고, “메인라인 머지”라는 말은 태그가 확인된 뒤에 쓴다.

6.2 나머지 두 패치

같은 날 patchwork를 다시 보니 3편과 4편의 패치는 둘 다 New였다. 자동 검사는 각각 17개가 전부 통과했고, lore 스레드에는 답장이 없다. 첫 패치가 Under Review로 바뀌기까지 9일이 걸린 것에 비하면 아직 늦었다고 볼 때가 아니다. 보낸 지 3주가 되는 10월 6-7일께까지 그대로면 같은 스레드에 짧게 재촉 메일을 보낸다.

한 장 요약

패치를 보낸 뒤 머지를 확인할 때 보는 것

보낸 다음 날

- ☐ lore에 원문이 올라왔는지 확인했다
- ☐ patchwork에 등록됐는지, 자동 검사가 모두 통과했는지 봤다

기다리는 동안

- ☐ patchwork 상태 변경 알림(New, Under Review, Accepted)의 뜻을 안다
- ☐ 리스트에 답장이 없어도 거절이 아니라는 것을 안다
- ☐ 재촉은 2-3주 뒤, 같은 스레드에 짧게 한다

받아들여진 뒤

- ☐ 커밋 알림 메일 제목에서 들어간 트리 와 브랜치를 읽었다
- ☐ git fetch 후 --author로 원격 브랜치에서 커밋을 찾았다
- ☐ patch-id로 보낸 것과 적용된 것이 같은지 비교했다
- ☐ 메인테이너가 붙인 Signed-off-by와 바뀐 해시를 기록했다
- ☐ 감사 답장은 리스트에 보내지 않았다

기록할 때

- ☐ 인용할 해시는 메인테이너 트리의 것으로 쓴다
- ☐ next에 들어간 것과 메인라인에 들어간 것을 구분해서 적는다
- ☐ 메인라인 반영은 릴리스 태그로 확인한 뒤에 적는다